



Daniel Macharia <danielmacharia43@gmail.com>

Batch 17 — Finance + P&L Deepening Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:21

Batch 17 — Finance + P&L Deepening Pack

Meat Lovers CIMS powered by YohPal

17A.

database/migrations/024_finance_categories.sql

```
CREATE TABLE finance_categories (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  category_name VARCHAR(255) NOT NULL,  
  
  category_type ENUM(  
    'INCOME',  
    'EXPENSE'  
  ) NOT NULL,  
  
  description TEXT,  
  
  is_active BOOLEAN DEFAULT TRUE,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

17B.

database/migrations/025_income_statement_snapshots.sql

```
CREATE TABLE income_statement_snapshots (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  statement_period ENUM(  
    'DAILY',  
    'WEEKLY',  
    'MONTHLY',  
    'ANNUAL'  
  ) NOT NULL,  
  
  period_start DATE NOT NULL,  
  period_end DATE NOT NULL,  
  
  food_sales DECIMAL(12,2) DEFAULT 0,  
  soft_drinks_sales DECIMAL(12,2) DEFAULT 0,  
  alcoholic_drinks_sales DECIMAL(12,2) DEFAULT 0,  
  
  total_sales DECIMAL(12,2) DEFAULT 0,  
  other_income DECIMAL(12,2) DEFAULT 0,  
  total_income DECIMAL(12,2) DEFAULT 0,  
  
  food_cost DECIMAL(12,2) DEFAULT 0,  
  soft_drinks_cost DECIMAL(12,2) DEFAULT 0,  
  alcoholic_drinks_cost DECIMAL(12,2) DEFAULT 0,  
  
  total_cost_of_goods DECIMAL(12,2) DEFAULT 0,  
  total_expenses DECIMAL(12,2) DEFAULT 0,  
  
  gross_profit DECIMAL(12,2) DEFAULT 0,  
  net_profit DECIMAL(12,2) DEFAULT 0,  
  
  created_by BIGINT,  
  
  FOREIGN KEY (created_by)  
    REFERENCES users(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```

```
);
```

17C.

database/migrations/026_cashier_reconciliations.sql

```
CREATE TABLE cashier_reconciliations (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  cashier_id BIGINT NOT NULL,

  reconciliation_date DATE NOT NULL,

  expected_cash DECIMAL(12,2) DEFAULT 0,
  declared_cash DECIMAL(12,2) DEFAULT 0,
  variance_amount DECIMAL(12,2) DEFAULT 0,

  expected_mpesa DECIMAL(12,2) DEFAULT 0,
  confirmed_mpesa DECIMAL(12,2) DEFAULT 0,
  mpesa_variance DECIMAL(12,2) DEFAULT 0,

  reconciliation_status ENUM(
    'PENDING',
    'BALANCED',
    'VARIANCE_FOUND',
    'APPROVED'
  ) DEFAULT 'PENDING',

  notes TEXT,

  created_by BIGINT,

  FOREIGN KEY (cashier_id)
    REFERENCES users(id),

  FOREIGN KEY (created_by)
    REFERENCES users(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

17D.

database/migrations/027_variance_alerts.sql

```
CREATE TABLE variance_alerts (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  variance_type ENUM(
    'CASH_SHORTAGE',
    'CASH_EXCESS',
    'MPESA_MISMATCH',
    'STOCK_VARIANCE',
    'SALES_VARIANCE'
  ) NOT NULL,

  reference_id BIGINT,

  variance_amount DECIMAL(12,2) DEFAULT 0,

  alert_status ENUM(
    'OPEN',
    'UNDER_REVIEW',
    'RESOLVED',
    'CLOSED'
  ) DEFAULT 'OPEN',

  notes TEXT,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

17E.

database/seeds/014_finance_categories_seed.sql

```
INSERT INTO finance_categories (
  category_name,
```

```

        category_type,
        description,
        is_active
    ) VALUES
    ('Food Sales', 'INCOME', 'Income from food sales', TRUE),
    ('Soft Drinks Sales', 'INCOME', 'Income from soft drinks sales', TRUE),
    ('Alcoholic Drinks Sales', 'INCOME', 'Income from alcoholic drinks sales', TRUE),
    ('Delivery Income', 'INCOME', 'Income from delivery charges', TRUE),
    ('Catering Income', 'INCOME', 'Income from catering jobs', TRUE),
    ('Other Income', 'INCOME', 'Other business income', TRUE),

    ('Food Stock Purchase', 'EXPENSE', 'Food ingredient and meat purchases', TRUE),
    ('Soft Drinks Purchase', 'EXPENSE', 'Soft drinks purchases', TRUE),
    ('Alcohol Purchase', 'EXPENSE', 'Alcoholic drinks purchases', TRUE),
    ('Staff Wages', 'EXPENSE', 'Staff wages and payroll', TRUE),
    ('Rent', 'EXPENSE', 'Premises rent', TRUE),
    ('Utilities', 'EXPENSE', 'Electricity, water, gas, and related utilities', TRUE),
    ('Transport', 'EXPENSE', 'Transport and logistics expenses', TRUE),
    ('Repairs and Maintenance', 'EXPENSE', 'Repairs, maintenance, and equipment servicing', TRUE),
    ('Marketing', 'EXPENSE', 'Marketing and customer acquisition expenses', TRUE),
    ('Miscellaneous Expense', 'EXPENSE', 'Other operating expenses', TRUE);

```

17F. Update

backend/routes/api.php

Add routes:

```

$router->get('/api/finance/categories', [FinanceController::class, 'categories']);
$router->post('/api/finance/categories', [FinanceController::class, 'storeCategory']);

$router->get('/api/finance/profitability-split', [FinanceController::class, 'profitabilitySplit']);

$router->post('/api/finance/reconciliations', [FinanceController::class, 'storeReconciliation']);
$router->get('/api/finance/reconciliations', [FinanceController::class, 'reconciliations']);

$router->get('/api/finance/variance-alerts', [FinanceController::class, 'varianceAlerts']);

$router->post('/api/income-statement/snapshot/daily', [IncomeStatementController::class, 'snapshotDaily']);
$router->post('/api/income-statement/snapshot/weekly', [IncomeStatementController::class, 'snapshotWeekly']);
$router->post('/api/income-statement/snapshot/monthly', [IncomeStatementController::class, 'snapshotMonthly']);
$router->post('/api/income-statement/snapshot/annual', [IncomeStatementController::class, 'snapshotAnnual']);
$router->get('/api/income-statement/snapshots', [IncomeStatementController::class, 'snapshots']);

```

17G. Update

backend/app/Controllers/FinanceController.php

Add these methods inside the class:

```

public function categories(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        'SELECT * FROM finance_categories ORDER BY category_type, category_name ASC'
    );

    Response::success(['finance_categories' => $stmt->fetchAll()]);
}

public function storeCategory(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO finance_categories
        (category_name, category_type, description, is_active)
        VALUES (:name, :type, :description, 1)'
    );

    $stmt->execute([
        'name' => $data['category_name'],
        'type' => $data['category_type'],
        'description' => $data['description'] ?? null,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'FINANCE',
        'CREATE_FINANCE_CATEGORY',

```

```

        null,
        null,
        $data
    );

    Response::success([], 'Finance category created');
}

public function profitabilitySplit(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            p.product_category,
            SUM(oi.total_price) AS sales,
            SUM(oi.quantity * p.cost_price) AS cost,
            SUM(oi.total_price) - SUM(oi.quantity * p.cost_price) AS gross_profit
        FROM order_items oi
        JOIN products p ON p.id = oi.product_id
        JOIN orders o ON o.id = oi.order_id
        WHERE o.order_status = 'PAID'
        GROUP BY p.product_category"
    );

    Response::success(['profitability_split' => $stmt->fetchAll()]);
}

public function storeReconciliation(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT', 'CASHIER']);
    $data = Request::body();

    $expectedCash = (float) ($data['expected_cash'] ?? 0);
    $declaredCash = (float) ($data['declared_cash'] ?? 0);
    $expectedMpesa = (float) ($data['expected_mpesa'] ?? 0);
    $confirmedMpesa = (float) ($data['confirmed_mpesa'] ?? 0);

    $cashVariance = $declaredCash - $expectedCash;
    $mpesaVariance = $confirmedMpesa - $expectedMpesa;

    $status = ($cashVariance == 0.0 && $mpesaVariance == 0.0)
        ? 'BALANCED'
        : 'VARIANCE_FOUND';

    $stmt = Database::connection()->prepare(
        'INSERT INTO cashier_reconciliations
        (cashier_id, reconciliation_date, expected_cash, declared_cash, variance_amount,
        expected_mpesa, confirmed_mpesa, mpesa_variance, reconciliation_status, notes, created_by)
        VALUES
        (:cashier_id, :reconciliation_date, :expected_cash, :declared_cash, :variance_amount,
        :expected_mpesa, :confirmed_mpesa, :mpesa_variance, :status, :notes, :created_by)'
    );

    $stmt->execute([
        'cashier_id' => $data['cashier_id'],
        'reconciliation_date' => $data['reconciliation_date'],
        'expected_cash' => $expectedCash,
        'declared_cash' => $declaredCash,
        'variance_amount' => $cashVariance,
        'expected_mpesa' => $expectedMpesa,
        'confirmed_mpesa' => $confirmedMpesa,
        'mpesa_variance' => $mpesaVariance,
        'status' => $status,
        'notes' => $data['notes'] ?? null,
        'created_by' => $user['id'],
    ]);

    $reconciliationId = (int) Database::connection()->lastInsertId();

    if ($cashVariance != 0.0) {
        $this->createVarianceAlert(
            $cashVariance < 0 ? 'CASH_SHORTAGE' : 'CASH_EXCESS',
            $reconciliationId,
            $cashVariance,
            'Cash reconciliation variance detected'
        );
    }

    if ($mpesaVariance != 0.0) {
        $this->createVarianceAlert(
            'MPESA_MISMATCH',
            $reconciliationId,
            $mpesaVariance,
            'M-Pesa reconciliation variance detected'
        );
    }
}

```

```

        AuditLogger::log(
            (int) $user['id'],
            'FINANCE',
            'CREATE_CASHIER_RECONCILIATION',
            $reconciliationId,
            null,
            $data
        );

        Response::success([
            'reconciliation_id' => $reconciliationId,
            'status' => $status,
        ], 'Cashier reconciliation recorded');
    }

    public function reconciliations(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

        $stmt = Database::connection()->query(
            "SELECT
                cr.*,
                cashier.full_name AS cashier_name,
                creator.full_name AS created_by_name
            FROM cashier_reconciliations cr
            JOIN users cashier ON cashier.id = cr.cashier_id
            LEFT JOIN users creator ON creator.id = cr.created_by
            ORDER BY cr.created_at DESC"
        );

        Response::success(['cashier_reconciliations' => $stmt->fetchAll()]);
    }

    public function varianceAlerts(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

        $stmt = Database::connection()->query(
            'SELECT * FROM variance_alerts ORDER BY created_at DESC'
        );

        Response::success(['variance_alerts' => $stmt->fetchAll()]);
    }

    private function createVarianceAlert(
        string $varianceType,
        int $referenceId,
        float $amount,
        string $notes
    ): void {
        $stmt = Database::connection()->prepare(
            'INSERT INTO variance_alerts
                (variance_type, reference_id, variance_amount, notes)
            VALUES (:type, :reference_id, :amount, :notes)'
        );

        $stmt->execute([
            'type' => $varianceType,
            'reference_id' => $referenceId,
            'amount' => $amount,
            'notes' => $notes,
        ]);
    }
}

```

17H. Update

backend/app/Controllers/IncomeStatementController.php

Add these methods inside the class:

```

    public function snapshotDaily(): void
    {
        $this->snapshot('DAILY', date('Y-m-d'), date('Y-m-d'));
    }

    public function snapshotWeekly(): void
    {
        $start = date('Y-m-d', strtotime('monday this week'));
        $end = date('Y-m-d', strtotime('sunday this week'));

        $this->snapshot('WEEKLY', $start, $end);
    }
}

```

```

public function snapshotMonthly(): void
{
    $start = date('Y-m-01');
    $end = date('Y-m-t');

    $this->snapshot('MONTHLY', $start, $end);
}

public function snapshotAnnual(): void
{
    $start = date('Y-01-01');
    $end = date('Y-12-31');

    $this->snapshot('ANNUAL', $start, $end);
}

public function snapshots(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        'SELECT * FROM income_statement_snapshots ORDER BY created_at DESC'
    );

    Response::success(['income_statement_snapshots' => $stmt->fetchAll()]);
}

private function snapshot(string $period, string $start, string $end): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $db = Database::connection();

    $sales = $this->salesSplit($start, $end);
    $otherIncome = $this->otherIncome($start, $end);
    $expenses = $this->expenses($start, $end);

    $foodSales = (float) ($sales['FOOD']['sales'] ?? 0);
    $softSales = (float) ($sales['SOFT_DRINK']['sales'] ?? 0);
    $alcoholSales = (float) ($sales['ALCOHOLIC_DRINK']['sales'] ?? 0);

    $foodCost = (float) ($sales['FOOD']['cost'] ?? 0);
    $softCost = (float) ($sales['SOFT_DRINK']['cost'] ?? 0);
    $alcoholCost = (float) ($sales['ALCOHOLIC_DRINK']['cost'] ?? 0);

    $totalSales = $foodSales + $softSales + $alcoholSales;
    $totalCost = $foodCost + $softCost + $alcoholCost;
    $totalIncome = $totalSales + $otherIncome;
    $grossProfit = $totalSales - $totalCost;
    $netProfit = $totalIncome - $totalCost - $expenses;

    $stmt = $db->prepare(
        'INSERT INTO income_statement_snapshots
        (statement_period, period_start, period_end,
        food_sales, soft_drinks_sales, alcoholic_drinks_sales,
        total_sales, other_income, total_income,
        food_cost, soft_drinks_cost, alcoholic_drinks_cost,
        total_cost_of_goods, total_expenses, gross_profit, net_profit, created_by)
        VALUES
        (:period, :start, :end,
        :food_sales, :soft_sales, :alcohol_sales,
        :total_sales, :other_income, :total_income,
        :food_cost, :soft_cost, :alcohol_cost,
        :total_cost, :expenses, :gross_profit, :net_profit, :created_by)'
    );

    $stmt->execute([
        'period' => $period,
        'start' => $start,
        'end' => $end,
        'food_sales' => $foodSales,
        'soft_sales' => $softSales,
        'alcohol_sales' => $alcoholSales,
        'total_sales' => $totalSales,
        'other_income' => $otherIncome,
        'total_income' => $totalIncome,
        'food_cost' => $foodCost,
        'soft_cost' => $softCost,
        'alcohol_cost' => $alcoholCost,
        'total_cost' => $totalCost,
        'expenses' => $expenses,
        'gross_profit' => $grossProfit,
        'net_profit' => $netProfit,
        'created_by' => $user['id'],
    ]);

    Response::success([], $period . ' income statement snapshot created');
}

```

```

}

private function salesSplit(string $start, string $end): array
{
    $stmt = Database::connection()->prepare(
        "SELECT
            p.product_category,
            SUM(oi.total_price) AS sales,
            SUM(oi.quantity * p.cost_price) AS cost
        FROM order_items oi
        JOIN products p ON p.id = oi.product_id
        JOIN orders o ON o.id = oi.order_id
        WHERE o.order_status = 'PAID'
        AND DATE(o.created_at) BETWEEN :start AND :end
        GROUP BY p.product_category"
    );

    $stmt->execute([
        'start' => $start,
        'end' => $end,
    ]);

    $rows = $stmt->fetchAll();
    $result = [];

    foreach ($rows as $row) {
        $result[$row['product_category']] = $row;
    }

    return $result;
}

private function otherIncome(string $start, string $end): float
{
    $stmt = Database::connection()->prepare(
        "SELECT COALESCE(SUM(amount), 0) AS total
        FROM finance_transactions
        WHERE transaction_type = 'INCOME'
        AND DATE(created_at) BETWEEN :start AND :end"
    );

    $stmt->execute([
        'start' => $start,
        'end' => $end,
    ]);

    $row = $stmt->fetch();

    return (float) $row['total'];
}

private function expenses(string $start, string $end): float
{
    $stmt = Database::connection()->prepare(
        "SELECT COALESCE(SUM(amount), 0) AS total
        FROM finance_transactions
        WHERE transaction_type = 'EXPENSE'
        AND DATE(created_at) BETWEEN :start AND :end"
    );

    $stmt->execute([
        'start' => $start,
        'end' => $end,
    ]);

    $row = $stmt->fetch();

    return (float) $row['total'];
}

```

171. Update

apps/admin-web/src/features/operations/api/operationsApi.js

Add methods:

```

financeCategories: async () => {
    const { data } = await apiClient.get('/finance/categories')
    return data.data.finance_categories
},

createFinanceCategory: async (payload) => {
    const { data } = await apiClient.post('/finance/categories', payload)
    return data
}

```

```

},

profitabilitySplit: async () => {
  const { data } = await apiClient.get('/finance/profitability-split')
  return data.data.profitability_split
},

createReconciliation: async (payload) => {
  const { data } = await apiClient.post('/finance/reconciliations', payload)
  return data
},

reconciliations: async () => {
  const { data } = await apiClient.get('/finance/reconciliations')
  return data.data.cashier_reconciliations
},

varianceAlerts: async () => {
  const { data } = await apiClient.get('/finance/variance-alerts')
  return data.data.variance_alerts
},

incomeStatementSnapshots: async () => {
  const { data } = await apiClient.get('/income-statement/snapshots')
  return data.data.income_statement_snapshots
},

snapshotDaily: async () => {
  const { data } = await apiClient.post('/income-statement/snapshot/daily', {})
  return data
},

snapshotWeekly: async () => {
  const { data } = await apiClient.post('/income-statement/snapshot/weekly', {})
  return data
},

snapshotMonthly: async () => {
  const { data } = await apiClient.post('/income-statement/snapshot/monthly', {})
  return data
},

snapshotAnnual: async () => {
  const { data } = await apiClient.post('/income-statement/snapshot/annual', {})
  return data
},

```

17J.

apps/admin-web/src/features/finance/pages/FinanceCategoriesPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function FinanceCategoriesPage() {
  return (
    <OperationalFormPage
      title="Finance Categories"
      subtitle="Manage income and expense categories"
      queryKey="finance-categories"
      queryFn={operationsApi.financeCategories}
      mutationFn={operationsApi.createFinanceCategory}
      initialForm={{
        category_name: '',
        category_type: 'EXPENSE',
        description: '',
      }}
      buildPayload={(form) => form}
      fields={[
        { name: 'category_name', label: 'Category Name' },
        {
          name: 'category_type',
          label: 'Category Type',
          type: 'select',
          options: [
            { value: 'INCOME', label: 'Income' },
            { value: 'EXPENSE', label: 'Expense' },
          ],
        },
        { name: 'description', label: 'Description', type: 'textarea' },
      ]}
      columns={[
        { key: 'category_name', label: 'Category' },

```

```

    { key: 'category_type', label: 'Type' },
    { key: 'description', label: 'Description' },
    { key: 'is_active', label: 'Active' },
  ]}
  />
)
}

```

17K.

apps/admin-web/src/features/finance/pages/ProfitabilitySplitPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function ProfitabilitySplitPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['profitability-split'],
    queryFn: operationsApi.profitabilitySplit,
  })

  return (
    <Page title="Profitability Split" subtitle="Food, soft drinks, and alcoholic drinks gross profit">
      <Card>
        {isLoading ? <LoadingBlock label="Loading profitability split..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'product_category', label: 'Category' },
              { key: 'sales', label: 'Sales' },
              { key: 'cost', label: 'Cost' },
              { key: 'gross_profit', label: 'Gross Profit' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}

```

17L.

apps/admin-web/src/features/finance/pages/CashierReconciliationPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function CashierReconciliationPage() {
  return (
    <OperationalFormPage
      title="Cashier Reconciliation"
      subtitle="Compare expected cash/M-Pesa against declared amounts"
      queryKey="cashier-reconciliations"
      queryFn={operationsApi.reconciliations}
      mutationFn={operationsApi.createReconciliation}
      initialForm={{
        cashier_id: '',
        reconciliation_date: '',
        expected_cash: '',
        declared_cash: '',
        expected_mpesa: '',
        confirmed_mpesa: '',
        notes: '',
      }}
    >
      buildPayload=({form}) => ({
        ...form,
        cashier_id: Number(form.cashier_id),
        expected_cash: Number(form.expected_cash),
        declared_cash: Number(form.declared_cash),
        expected_mpesa: Number(form.expected_mpesa),
        confirmed_mpesa: Number(form.confirmed_mpesa),
      })
    </OperationalFormPage>
  )
}

```

```

    { name: 'cashier_id', label: 'Cashier ID', type: 'number' },
    { name: 'reconciliation_date', label: 'Reconciliation Date', type: 'date' },
    { name: 'expected_cash', label: 'Expected Cash', type: 'number' },
    { name: 'declared_cash', label: 'Declared Cash', type: 'number' },
    { name: 'expected_mpesa', label: 'Expected M-Pesa', type: 'number' },
    { name: 'confirmed_mpesa', label: 'Confirmed M-Pesa', type: 'number' },
    { name: 'notes', label: 'Notes', type: 'textarea' },
  ]}
  columns=[
    { key: 'cashier_name', label: 'Cashier' },
    { key: 'reconciliation_date', label: 'Date' },
    { key: 'expected_cash', label: 'Expected Cash' },
    { key: 'declared_cash', label: 'Declared Cash' },
    { key: 'variance_amount', label: 'Cash Variance' },
    { key: 'expected_mpesa', label: 'Expected M-Pesa' },
    { key: 'confirmed_mpesa', label: 'Confirmed M-Pesa' },
    { key: 'mpesa_variance', label: 'M-Pesa Variance' },
    { key: 'reconciliation_status', label: 'Status' },
  ]}
  submitLabel="Record Reconciliation"
/>
)
}

```

17M.

apps/admin-web/src/features/finance/pages/VarianceAlertsPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function VarianceAlertsPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['variance-alerts'],
    queryFn: operationsApi.varianceAlerts,
  })

  return (
    <Page title="Variance Alerts" subtitle="Cash, M-Pesa, stock, and sales variance alerts">
      <Card>
        {isLoading ? <LoadingBlock label="Loading variance alerts..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns=[
              { key: 'variance_type', label: 'Type' },
              { key: 'reference_id', label: 'Reference' },
              { key: 'variance_amount', label: 'Amount' },
              { key: 'alert_status', label: 'Status' },
              { key: 'notes', label: 'Notes' },
              { key: 'created_at', label: 'Created' },
            ]
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}

```

17N.

apps/admin-web/src/features/finance/pages/IncomeStatementSnapshotsPage.jsx

```

import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function IncomeStatementSnapshotsPage() {
  const qc = useQueryClient()

```

```

const { data, isLoading, error } = useQuery({
  queryKey: ['income-statement-snapshots'],
  queryFn: operationsApi.incomeStatementSnapshots,
})

const mutation = useMutation({
  mutationFn: (period) => {
    if (period === 'daily') return operationsApi.snapshotDaily()
    if (period === 'weekly') return operationsApi.snapshotWeekly()
    if (period === 'monthly') return operationsApi.snapshotMonthly()
    return operationsApi.snapshotAnnual()
  },
  onSuccess: () => qc.invalidateQueries({ queryKey: ['income-statement-snapshots'] }),
})

return (
  <Page title="Income Statement Snapshots" subtitle="Saved daily, weekly, monthly, and annual P&L statements">
    {mutation.error ? <StatusMessage error={mutation.error.message} /> : null}

    <Card title="Create Snapshot">
      <div style={{ display: 'flex', gap: 10, flexWrap: 'wrap' }}>
        <Button onClick={() => mutation.mutate('daily')}>Daily Snapshot</Button>
        <Button onClick={() => mutation.mutate('weekly')}>Weekly Snapshot</Button>
        <Button onClick={() => mutation.mutate('monthly')}>Monthly Snapshot</Button>
        <Button onClick={() => mutation.mutate('annual')}>Annual Snapshot</Button>
      </div>
    </Card>

    <Card title="Saved Statements">
      {isLoading ? <LoadingBlock label="Loading snapshots..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {!isLoading && !error ? (
        <DataTable
          columns={[
            { key: 'statement_period', label: 'Period' },
            { key: 'period_start', label: 'Start' },
            { key: 'period_end', label: 'End' },
            { key: 'food_sales', label: 'Food Sales' },
            { key: 'soft_drinks_sales', label: 'Soft Drinks' },
            { key: 'alcoholic_drinks_sales', label: 'Alcohol' },
            { key: 'total_sales', label: 'Total Sales' },
            { key: 'total_expenses', label: 'Expenses' },
            { key: 'net_profit', label: 'Net Profit' },
            { key: 'created_at', label: 'Created' },
          ]}
          rows={data || []}
        />
      ) : null}
    </Card>
  </Page>
)
}

```

170. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```

import FinanceCategoriesPage from '../features/finance/pages/FinanceCategoriesPage'
import ProfitabilitySplitPage from '../features/finance/pages/ProfitabilitySplitPage'
import CashierReconciliationPage from '../features/finance/pages/CashierReconciliationPage'
import VarianceAlertsPage from '../features/finance/pages/VarianceAlertsPage'
import IncomeStatementSnapshotsPage from '../features/finance/pages/IncomeStatementSnapshotsPage'

```

Add routes inside DashboardLayout:

```

<Route path="/finance/categories" element={<FinanceCategoriesPage />} />
<Route path="/finance/profitability-split" element={<ProfitabilitySplitPage />} />
<Route path="/finance/reconciliations" element={<CashierReconciliationPage />} />
<Route path="/finance/variance-alerts" element={<VarianceAlertsPage />} />
<Route path="/finance/income-statement-snapshots" element={<IncomeStatementSnapshotsPage />} />

```

17P. Update

apps/admin-web/src/components/common/SidebarNav.jsx

Add links:

```
['Finance Categories', '/finance/categories'],
['Profitability Split', '/finance/profitability-split'],
['Cashier Reconciliation', '/finance/reconciliations'],
['Variance Alerts', '/finance/variance-alerts'],
['P&L Snapshots', '/finance/income-statement-snapshots'],
```

17Q.

docs/FINANCE_P_AND_L_DEEPENING_PACK.md

Batch 17 – Finance + P&L Deepening Pack

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch deepens finance from basic income and expenses into controlled P&L, category tracking, reconciliation, and variance detection.

Tables Added

1. `finance\_categories`
2. `income\_statement\_snapshots`
3. `cashier\_reconciliations`
4. `variance\_alerts`

Finance Categories

The system now separates:

Income

- Food Sales
- Soft Drinks Sales
- Alcoholic Drinks Sales
- Delivery Income
- Catering Income
- Other Income

Expenses

- Food Stock Purchase
- Soft Drinks Purchase
- Alcohol Purchase
- Staff Wages
- Rent
- Utilities
- Transport
- Repairs and Maintenance
- Marketing
- Miscellaneous Expense

P&L Snapshot Types

1. Daily
2. Weekly
3. Monthly
4. Annual

Each snapshot stores:

- food sales
- soft drinks sales
- alcoholic drinks sales
- total sales
- other income
- cost of goods
- total expenses
- gross profit
- net profit

Profitability Split

The system calculates profitability by:

- food
- soft drinks
- alcoholic drinks

Cashier Reconciliation

The system compares:

- expected cash vs declared cash
- expected M-Pesa vs confirmed M-Pesa

Variance Detection Placeholder

Variance alerts are created for:

- cash shortage
- cash excess
- M-Pesa mismatch

Future phases can extend variance detection to:

- stock variance
 - sales variance
 - bar variance
 - kitchen usage variance
-

Batch 17 Outcome

Finance + P&L now includes:

- ✓ expense categories
- ✓ income categories
- ✓ daily income statement table
- ✓ weekly income statement table
- ✓ monthly income statement table
- ✓ annual income statement table
- ✓ food / soft drinks / alcohol profitability split
- ✓ cashier settlement reconciliation
- ✓ variance detection placeholder

Next Smart Move

Build **Batch 18 — Staff Motivation + Service Performance Pack**, including:

- waiter sales leaderboard
- service team performance page
- customer rating capture
- bonus rules table
- daily waiter target tracking
- staff motivation dashboard
- HRM performance reports